



Ai registry mech

Security Review

Cantina Managed review by:

Saw-mon and Natalie, Lead Security Researcher
Slowfi, Security Researcher

February 19, 2025

Contents

1	Introduction	3
1.1	About Cantina	3
1.2	Disclaimer	3
1.3	Risk assessment	3
1.3.1	Severity Classification	3
2	Security Review Summary	4
3	Findings	5
3.1	Critical Risk	5
3.1.1	Anyone can call <code>adjustMechRequesterBalances</code>	5
3.2	High Risk	5
3.2.1	Make sure only the service id's multisig can call the <code>create</code> endpoint of the mech marketplace	5
3.2.2	Requesters can avoid paying mechs for the work performed when <code>deliverMarketplaceWithSignatures</code> is called	6
3.2.3	Requester credit burn can cause discounted delivery to revert	7
3.3	Medium Risk	9
3.3.1	Multiple Mech Marketplace proxies using the same implementation have the same domain separators	9
3.3.2	<code>deliverMarketplaceWithSignatures</code> can be frontrun	9
3.3.3	Use <code>safeTransfer</code> and <code>safeTransferFrom</code>	10
3.3.4	Request to mech marketplace can be frontrun by the mech operators to change <code>deliveryRate</code> and <code>paymentType</code>	11
3.3.5	Make sure requesters provide the payment type and also the max delivery rate to the request endpoints	11
3.3.6	Setting a balance tracker for a payment type which already has a balance tracker can cause issues	12
3.4	Low Risk	12
3.4.1	Event related issues	12
3.4.2	Karma can be gamed	13
3.4.3	Lack of two step ownership transfer pattern	13
3.4.4	Requests can be delivered non-sequentially	13
3.4.5	A mech calling <code>deliverMarketplaceWithSignatures</code> can use any payment type	14
3.4.6	Free deliveries are only allowed when delivering with signature	14
3.4.7	Missing validation for balance tracker in <code>_requestBatch</code>	14
3.4.8	Delivering with signature does not check that <code>requestDatas[i]</code> should have a positive length	15
3.5	Gas Optimization	15
3.5.1	memory can be replace by <code>calldata</code> to save gas	15
3.5.2	Owner role only used for initializing subscription parameters	16
3.5.3	Unused <code>paymentData</code> parameter	16
3.6	Informational	16
3.6.1	Checking array lengths match is missing	16
3.6.2	<code>getRequestStatus</code> can be simplified	17
3.6.3	Use EIP-712 to derive request ids	17
3.6.4	Canceling requests	17
3.6.5	Allow requesters to withdraw unused balances	18
3.6.6	Unused code, Typos, Comments...	18
3.6.7	Implementation contracts can be initialized	18
3.6.8	Burning or transferring <code>subscriptionNFT</code> credits should not be allowed	19
3.6.9	To avoid checking array lengths for related arrays use structs	20
3.6.10	<code>mapRequestIdFinalizedRates</code> values cannot be trusted	20
3.6.11	Don't revert when a result is delivered by a different mech than the priority one during the response timeout period	21
3.6.12	Mech operators can have the mech call any contract or perform any custom logic	21
3.6.13	<code>setUp</code> function visibility can be restricted	22
3.6.14	Interface recommendations	23

3.6.15 When minting not subscription credit make sure the user always provides non-zero msg.value	23
3.6.16 Best practices on signature verification process	23
3.6.17 Usage of modifiers	24
3.6.18 Consistent getImplementation function placement across proxies	24
3.6.19 Store _serviceRegistry and _serviceId as immutable parameters in the OlasMech contract	24
3.6.20 Update responseTimeout and paymentType when delivering requests with signature .	25
3.6.21 _verifySignedHash does not catch incorrect v values early	25
3.6.22 No blocking mechanism for malicious mechs in the marketplace exists	27

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	<i>Must fix as soon as possible (if already deployed).</i>
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Valory is a research and deployment company at the intersection of crypto and AI. Specifically, the premier creator of open-source frameworks for co-owned AI. Valory's mission is to enable communities, organizations, and countries to co-own AI systems, beginning with decentralized autonomous agents. Valory is the VC-backed team of engineers, researchers, and commercial executors that co-founded the Olas DAO, contribute to the Olas Protocol and Olas Stack, and built the first agents using it.

From Jan 27th to Feb 6th the Cantina team conducted a review of [ai-registry-mech](#) on commit hash [71cca93](#). The team identified a total of **43** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	1	1	0
High Risk	3	2	1
Medium Risk	6	4	2
Low Risk	8	5	3
Gas Optimizations	3	1	2
Informational	22	12	10
Total	43	25	18

The Cantina Managed team reviewed [ai-registry-mech](#) changes holistically on commit hash [4273fa34](#) and concluded that all issues were addressed and no new vulnerabilities were introduced.

3 Findings

3.1 Critical Risk

3.1.1 Anyone can call `adjustMechRequesterBalances`

Severity: Critical Risk

Context: [BalanceTrackerBase.sol#L295-L328](#)

Description: Unlike `checkAndRecordDeliveryRates` and `finalizeDeliveryRates`, `adjustMechRequesterBalances` is missing the check to make sure only the `mechMarketplace` can call this endpoint. Anyone can call this endpoint with arbitrary parameters and can manipulate all the `mapRequesterBalances` and `mapMechBalances` and thus drain all funds from this contract.

Proof of Concept: add the following test case to `test/MechFixedPriceToken.js` and run:

```
npm exec hardhat test -- --grep "Balance Tracker"

context("Balance Tracker", async function () {
  it("Anyone can call adjustMechRequesterBalances", async function () {
    const bob = signers[1];
    const bt = await ethers.getContractAt("BalanceTrackerFixedPriceToken",
      ↪ balanceTrackerFixedPriceToken.address);

    await token.approve(balanceTrackerFixedPriceToken.address, initMint);
    await bt.deposit(initMint);

    await bt.connect(bob).adjustMechRequesterBalances(
      bob.address,
      deployer.address,
      [initMint],
      "0x"
    );

    const bobBalance = await bt.mapMechBalances(bob.address);
    const deployerBalance = await bt.mapRequesterBalance(deployer.address);

    expect(bobBalance).to.be.equal(initMint);
    expect(deployerBalance).to.be.equal(0);
  });
});
```

Recommendation: Make sure the following check is included in this function:

```
// Check for marketplace access
if (msg.sender !== mechMarketplace) {
  revert MarketplaceOnly(msg.sender, mechMarketplace);
}
```

Valory: Fixed on [PR 90](#).

Cantina Managed: Fix verified. The fix checks that only the marketplace can call this external function.

3.2 High Risk

3.2.1 Make sure only the service id's multisig can call the `create` endpoint of the mech marketplace

Severity: High Risk

Context: [MechMarketplace.sol#L437-L458](#)

Description: Anyone can call the `create` endpoint of the mech market and with any:

- `serviceId`.
- `payload`.

for any allowed `mechFactory`. This has the following consequences:

- `mapServiceIdMech[serviceId]` can be overridden for any previously created mech and thus cause the `checkMech` function to fail for those old mechs. This would cause all the deliver endpoints called by the old mech with the same service id to fail.
- Since for each request, the information stored only include the mech addresses (priority and delivery mechs) and not the service ids, the priority mech would not be able to deliver the request.

The above can be used to:

- DoS the delivery endpoints of the marketplace.
- block the priority mech of a request from delivery the request and thus after the `responseTimeout` has been passed any other mech would be able to deliver the request.
- set the payload upon creation which would affect parameters such as max delivery rate.
- register/create the mech with the different allowed mech factory with a different payment type.

Proof of Concept: add the following test case to `test/MechFixedPriceNative.js` and run:

```
npm exec hardhat test -- --grep "Creating Mech for the same service id"

context("Creating Mech for the same service id", async function () {
  async function createMech(user, mechServiceID, maxDeliveryRate) {
    const mechCreationData = ethers.utils.defaultAbiCoder.encode(["uint256"], [maxDeliveryRate]);
    const tx = await mechMarketplace.connect(user).create(mechServiceID, mechFactoryFixedPrice.address,
      ↪ mechCreationData);
    const res = await tx.wait();
    // Get mech contract address from the event
    const mechAddress = "0x" + res.logs[0].topics[1].slice(26);
    // Get mech contract instance
    return await ethers.getContractAt("MechFixedPriceNative", mechAddress);
  }

  it("same service id", async function () {
    const bob = signers[1];

    const serviceId = await priorityMech.tokenId();
    const multiSig = await priorityMech.getOperator();
    const ms = await mechMarketplace.checkMech(priorityMechAddress);
    await expect(
      ms
    ).to.be.equal(multiSig);

    // bob creates a mech with the same service id as the priority mech and
    // sets the max delivery rate to 10
    const auxMech = await createMech(bob, serviceId, 10);

    // `checkMech` would revert and that means the priority mech cannot deliver any requests
    await expect(
      mechMarketplace.checkMech(priorityMechAddress)
    ).to.be.revertedWithCustomError(mechMarketplace, "UnauthorizedAccount");

  });
});
```

Recommendation: Make sure the create endpoint can only be called by the multisig of the service id in the service registry.

Valory: Fixed in commit [513fd8e8](#).

Cantina Managed: Fix verified.

3.2.2 Requesters can avoid paying mechs for the work performed when `deliverMarketplaceWithSignatures` is called

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: Requesters can avoid paying mechs for the work performed when `deliverMarketplaceWithSignatures` is called. Assume a requester signs multiple requests $\{R_1, \dots, R_m\}$ with nonces $\{n_1, \dots, n_m\}$ and its current nonce is n_1 . A mech M picks up these requests and performs the necessary

tasks to calculate the `deliveryDatas` and then calls the `deliverMarketplaceWithSignatures` with the necessary parameters (call this tx T). Here are ways the `requester` can avoid paying for M's services and still use `deliveryDatas`:

The `requester` look at the public mempool and sees M's transaction and thus reads `deliveryDatas`. Then to avoid paying it would either:

- Make sure its balance in the balance tracker is 0 and thus T would fail, or...
- Frontrun T, create fake requests and deliver them in the same transaction to pull its funds (minus the marketplace fees) out of the balance tracker. Or.
- Frontrun T, create fake requests with almost 0 max delivery rates (free deliveries) to **use up the nonces** $\{n_1, \dots, n_m\}$. The `requester` don't even have to deliver them. This will invalidate the signatures that it had already provided for the requests M wants to deliver, since the **nonces used/checked** in `deliverMarketplaceWithSignatures` would start from $n_m + 1$.
- Even without front-running, if the `requester` has a non-zero code length it can at will revert T whenever during the signature verification it gets the `ISignatureValidator(requester).isValidSignature(...)` callback. Note that after Pectra upgrade even EOAs can have delegations which could perform this on their behalf.

Recommendation:

1. Regarding the balance issues perhaps `requesters` would need to commit balances/credits to `mech` or service ids (service ids might be better since a `mech` for service id could be updated). So when sending funds to balance trackers you can either do a general deposit or deposits that are directed towards specific services. This might also create an issue since for example the `mechs` might stop being functional and thus not performing requests and thus those funds would be locked and would not be able to be used by other `mechs`. In addition to this and maybe to avoid the issue with non-operational `mechs`, it might also make sense to introduce a locking/vesting period where after a certain period of time the balances that are unused by the non-operational `mech` could be added to the `requester`'s general balance.
2. Regarding the `nonce` issue, it would make sense for the `requester` to commit to those nonces beforehand either by calling an endpoint of the market place directly or providing signatures (to avoid paying for gas) to a 3rd party which can call the marketplace on their behalf. Committing would mean basically the endpoint increasing the `mapNonces[requester]`. Moreover the marketplace would need to have two mapping for the nonces, one to track committed nonces by the `requester` and one to track delivered nonces by the `mechs` both would have the same `requester` as the key.

Valory: The scenario outlined is accepted as a reasonable tradeoff.

`Mechs` when receiving off-chain requests should do the following:

1. Ensure that all signatures are valid.
2. The delivery `mech` should verify that the delivery will be finalized on-chain if submitted against the current state. One way to do it is for delivery `mechs` to perform the gas estimate, and if failed - not proceed with requested delivery. In order to save on compute, the delivery gas estimation can be done with mocks of delivery data first to make sure the `requester` has correct balances to cover.
3. Delivery `mechs` need to be aware that `requesters` might request from multiple `mechs` at the same time off-chain and therefore cause resource contention for the available funds. It is the responsibility of the delivery `mech` to track the reputation of `requesters` and accordingly accept or reject off-chain work.

From on-chain changes, to enhance the protection of delivery with signatures, we can add the priority `mech` address into the request id hashing.

Cantina Managed: Acknowledged by Valory team and added to documentation.

3.2.3 Requester credit burn can cause discounted delivery to revert

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: The function `redeemRequesterCredits` in `BalanceTrackerNvmSubscriptionNative.sol` allows a requester to burn their available credits at any time. If this function is called after submitting a request but before the mech delivers it, the delivery operation reverts due to a balance underflow check in `_adjustFinalBalance`.

This occurs when the mech applies a discounted delivery rate. The function `finalizeDeliveryRates` computes a difference (`rateDiff`) between the originally expected rate and the mech's discounted rate. If the requester has burned their credits in the meantime, their balance is set to zero, leading to a revert when `_adjustFinalBalance` checks for sufficient balance. As a result, the request cannot be finalized, preventing successful delivery.

Proof of Concept: The following test case demonstrates the issue:.

```
function test_sendRequestNevermined() public {
    bytes32 requestId;
    {
        erc1155.mint(address(this), uint256(1), 10 ether, new bytes(0));
    }
    {
        serviceRegistry.setServiceOwner(uint32(3), address(this));
        serviceRegistry.setServiceOwner(uint32(4), address(this));
        address mech = marketPlace.create(uint256(3), address(mechFactoryNvm), abi.encode(maxDeliveryRate));
        mechsNvm.push(MechNvmSubscriptionNative payable(mech));
    }
    {
        bytes memory requestData = new bytes(1);
        uint256 priorityMechServiceId = uint256(3);
        uint256 responseTimeout = maxResponseTimeout - 100;
        bytes memory paymentData;
        requestId = marketPlace.request(requestData, priorityMechServiceId, responseTimeout, paymentData);
    }
    {
        bytes32
        bytes
        requestIds[0] = requestId;
        deliveryDatas[0] = abi.encode(0.5 ether, new bytes(0));

        // Burn credits before delivery
        balanceNvm.redeemRequesterCredits(address(this)); // If commented, delivery succeeds

        // Attempt to deliver the request
        mechsNvm[0].deliverToMarketplace(requestIds, deliveryDatas);
    }
}
```

If the burn operation is executed before the delivery, the delivery reverts due to an underflow in `_adjustFinalBalance`. And moreover, an excess credit is burned on the call to `redeemRequesterCredits`.

Recommendation: The credit-burning mechanism could consider pending requests to ensure that balance updates do not interfere with service delivery. One possible approach is restricting `redeemRequesterCredits` from being called while there are active requests or ensuring that burned credits do not affect outstanding deliveries. Another approach could involve modifying how delivery rate adjustments interact with balance updates, preventing unintended reverts while maintaining flexibility in credit usage.

Valory: Fixed on [PR 99](#).

The new workflow that removes that issue and solves the #18 one (Low) perform the next steps:

- When the request is made - burn requester credits in the amount of total request cost minus the leftover credits from previous discounted delivery actual rates vs max delivery rate.
- When the delivery is made - record any discounts of deliveries into the discount balance for the requester. This way the discount balance only collects leftovers from delivery discounts, and can be adjusted to the total request cost when a new request is made, as described above.
- If the requester burns credits themselves at any point of time - it would simply reflect its possibility to post more requests, without invalidating the math of already requested ones.

Cantina Managed: Fix verified.

3.3 Medium Risk

3.3.1 Multiple Mech Marketplace proxies using the same implementation have the same domain separators

Severity: Medium Risk

Context: [MechMarketplace.sol#L152-L167](#)

Description: Currently in MechMarketplace, the immutable variable domainSeparator is set in the constructor. This causes all the proxies using this implementation to have the same domainSeparator which would allow signature replays across multiple mech marketplace proxies on the same chain.

Proof of Concept: add the following test to `test/MechMarketplace.js` and run:

```
npm exec hardhat test -- --grep "Multiple Mech"
```

```
context("Multiple Mech Market Proxies with the same original implementation", async function () {
  let m1;
  let m2;

  beforeEach(async function () {
    const MechMarketplace = await ethers.getContractFactory("MechMarketplace");
    const mechMarketplace = await MechMarketplace.deploy(serviceRegistry.address, karma.address);
    await mechMarketplace.deployed();

    async function deployMechMarketplaceProxy() {
      // Deploy and initialize marketplace proxy
      const proxyData = MechMarketplace.interface.encodeFunctionData("initialize",
        [fee, minResponseTimeout, maxResponseTimeout]);
      const MechMarketplaceProxy = await ethers.getContractFactory("MechMarketplaceProxy");
      const mechMarketplaceProxy = await MechMarketplaceProxy.deploy(mechMarketplace.address, proxyData);
      await mechMarketplaceProxy.deployed();

      return ethers.getContractAt("MechMarketplace", mechMarketplaceProxy.address);
    }

    m1 = await deployMechMarketplaceProxy();
    m2 = await deployMechMarketplaceProxy();
  });

  it("Domain Separators should not match", async function() {
    const d1 = await m1.getDomainSeparator();
    const d2 = await m2.getDomainSeparator();

    // two different mech markets proxies with the same shared original implementation
    // should not have the same domain separators
    expect(d1).not.to.equal(d2);
  });
});
```

Recommendation: Make sure domainSeparator is initiated in during initialisation and not in the constructor. Also initialisation for proxy contracts is recommended to be blocked.

Valory: Fixed on [PR 93](#).

Cantina Managed: Fix verified.

3.3.2 deliverMarketplaceWithSignatures can be frontrun

Severity: Medium Risk

Context: [MechMarketplace.sol#L732-L733](#)

Description: When a requester signs requests to be delivered by a priority mech through `deliverMarketplaceWithSignatures`, the request/signature and its verification process does not bind the request to a specific mech.

Assume a mech M_1 has in possession a request of a requester and its corresponding signature, then it performs a very resource intensive task to deliver the delivery data for the requester. When M_1 calls the `deliverMarketplaceWithSignatures` endpoint of the marketplace while its transaction is in the mempool

(public mempools), any other mech M_2 can frontrun this transaction to deliver the computed data and in returns:

- Increase its karma for M_2 .
- Steal the payment from M_1 .

Proof of Concept: add the following test case to test/MechFixedPriceNative.js and run:

```
npm exec hardhat test -- --grep "Frontrun requests with signatures"
```

```
context("Deliver", async function () {
  it("Frontrun requests with signatures", async function () {
    const numRequests = 10;
    const datas = new Array();
    const requestIds = new Array();
    const signatures = new Array();
    const deliveryRates = new Array(numRequests).fill(maxDeliveryRate);
    let requestCount = 0;

    // Pre-pay the contract insufficient amount for posting a request
    await deployer.sendTransaction({to: balanceTrackerFixedPriceNative.address, value: maxDeliveryRate *
    ↪ numRequests});

    // Get deployer wallet
    const accounts = config.networks.hardhat.accounts;
    const wallet = ethers.Wallet.fromMnemonic(accounts.mnemonic, accounts.path + `/${0}`);
    const signingKey = new ethers.utils.SigningKey(wallet.privateKey);

    // Stack all requests
    for (let i = 0; i < numRequests; i++) {
      datas[i] = data + "00".repeat(i);
      requestIds[i] = await mechMarketplace.getRequestId(deployer.address, datas[i], maxDeliveryRate,
      ↪ requestCount);
      const signature = signingKey.signDigest(requestIds[i]);
      // Extract v, r, s
      const r = ethers.utils.arrayify(signature.r);
      const s = ethers.utils.arrayify(signature.s);
      const v = ethers.utils.arrayify(signature.v);
      // Assemble 65 bytes of signature
      signatures[i] = ethers.utils.hexlify(ethers.utils.concat([r, s, v]));
      requestCount++;
    }

    // an expensive task performed by the priority mech
    async function expensiveComputation(requestDatas) {
      return requestDatas;
    }

    const deliveryDatas = await expensiveComputation(datas);

    // different mech delivers the request
    await deliveryMech.deliverMarketplaceWithSignatures(deployer.address, datas, signatures, deliveryDatas,
    deliveryRates, "0x");

    // Deliver requests by priority mech fails
    await expect(
      priorityMech.deliverMarketplaceWithSignatures(deployer.address, datas, signatures, deliveryDatas,
      deliveryRates, "0x")
    ).to.be.revertedWithCustomError(mechMarketplace, "SignatureNotValidated");
  });
});
```

Recommendation: Make sure the derived requestId hash take into consideration the priority mech address among other values.

Valory: Fixed in commit [f953c1e5](#).

Cantina Managed: Fix verified.

3.3.3 Use safeTransfer and safeTransferFrom

Severity: Medium Risk

Context: [BalanceTrackerFixedPriceNative.sol#L12](#), [BalanceTrackerFixedPriceNative.sol#L45](#),

BalanceTrackerFixedPriceToken.sol#L12, BalanceTrackerFixedPriceToken.sol#L19, BalanceTrackerFixedPriceToken.sol#L50, BalanceTrackerFixedPriceToken.sol#L72, BalanceTrackerFixedPriceToken.sol#L84, BalanceTrackerFixedPriceToken.sol#L96, BalanceTrackerFixedPriceToken.sol#L110

Description: In BalanceTrackerFixedPriceNative and BalanceTrackerFixedPriceToken, the IToken interface has been used that defines transfer and transferFrom functions to return a bool.

1. Not all tokens conform to this interface when transferring amounts.
2. The returned bool value has not been checked.

Recommendation: Use the safer implementation of token transfers to rule out the above issues. Look at OpenZeppelin's implementation of [SafeERC20](#).

Valory: Acknowledged. New token contract can be created for these cases if required.

Cantina Managed: Only comments have been added in commit [652256e9](#) warning about this issue.

3.3.4 Request to mech marketplace can be frontrun by the mech operators to change deliveryRate and paymentType

Severity: Medium Risk

Context: [MechMarketplace.sol#L232-L245](#)

Description: When calling the request endpoints of the mech marketplace, the requester only provides the priorityMechServiceId. Then the deliveryRate and paymentType are fetched from the corresponding priorityMech. This mech's operator can frontrun the requests call and:

- Create a new mech for the same service id but with a different mech factory / payment type.
- Change the max delivery rate of the mech.

And thus force the requester to use potentially unwanted settings for its request.

Recommendation: To avoid the above issues make sure that the requester can provide the payment types and delivery rates to the request endpoints which can later be matched/compared with the ones of the priorityMech.

Valory: Fixed in commit [370e2b72](#).

Cantina Managed: Fix verified.

3.3.5 Make sure requesters provide the payment type and also the max delivery rate to the request endpoints

Severity: Medium Risk

Context: [MechMarketplace.sol#L241-L245](#)

Description: Currently when a request is create by a requester the deliveryRate and paymentType are fetched from the corresponding priorityMech of a service id. Just before this transaction get published those values could change and potentially cause harm to the requester. For example if the request was for a token mech and the requester had approved the token balance tracker infinite approval for the token, just before the call the mech operator could change the max delivery rate to the token balance of the user and thus pull out all the requester's fund upon delivery of the request.

Recommendation: To protect against the above issues, it would be best if the requester could provide those values to the request endpoints which later can be compared against the values fetched from the priorityMech:

```
// check
requesterPaymentType == mechPaymentType
requesterMaxDeliveryRate >= mechMaxDeliveryRate
```

Valory: Fixed in commit [370e2b72](#).

Cantina Managed: Fix verified.

3.3.6 Setting a balance tracker for a payment type which already has a balance tracker can cause issues

Severity: Medium Risk

Context: MechMarketplace.sol#L488-L514

Description: Setting a balance tracker for a payment type which already has a balance tracker can cause the following issue:

- If some requesters have unused credit for the old balance tracker of that payment type after the switch to the new balance tracker, they would not be able to use their unused credits (unless those parameters are also migrated to the new balance tracker).
- The mechs who have already performed some of the requests using the payment type in question when trying to deliver to the marketplace upon balance tracker accounting callback, after the deliver call they might not be able to withdraw their credits because the new balance tracker might not have enough underlying assets. Or if the delivery to the marketplace is with signatures, the requesters might have not approved the new balance tracker to pull tokens (or it might not have native tokens or NFT subscriptions, ...).

Recommendation: Due to the above issues the marketplace owner must be very careful one updating a balance tracker for a payment type that already has a balance tracker or revert if `mapPaymentTypeBalanceTrackers[paymentTypes[I]]` is already a non-zero address.

Valory: Acknowledged as a design decision.

Cantina Managed: Acknowledged by Valory team.

3.4 Low Risk

3.4.1 Event related issues

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description/Recommendation:

- Missing events:
 - MechMarketplace.sol#L369: MarketplaceParamsUpdated is not emitted . Best to move this event in `_changeMarketplaceParams`. Note that the event is emitted when `changeMarketplaceParams` is called.
 - MechMarketplace.sol#L371: OwnerUpdated not emitted.
 - BalanceTrackerBase.sol#L83-L84.
- Others:
 - OIasMech.sol#L245-L252: Have the mech marketplace to emit Deliver and RevokeRequest so that the event emission upon delivery is unified for both with and without signatures.
 - OIasMech.sol#L247: Related to above and also let Deliver to emit `deliveryRates[i]`.
 - BalanceTrackerFixedPriceNative.sol#L79: Should be `Withdraw(account, ...)`.
 - BalanceTrackerFixedPriceToken.sol#L86: Should be `Withdraw(account, ...)`.
 - BalanceTrackerBase.sol#L284: Perhaps it should also emit `totalRequesterDeliveryRate` since currently this value has not been used.

Valory: Fixed in commit 554fd383.

Cantina Managed: Fix verified.

3.4.2 Karma can be gamed

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: Any entity operating a mech can create simple requests and set the `mech` they are operating as the priority mech and have the mech deliver right away and thus increase its `mapMechKarma`.

On the other hand `mapRequesterMechKarma` for a specific requester would be harder to manipulate since if the mech does not deliver quality data then the requester would stop creating requests that would have that specific mech as the priority mech (linked by the service id). Although this can also be manipulated if one considers the fronrunning issues related to `deliverMarketplaceWithSignatures`.

Recommendation: A monitoring and analysis should be setup to shine a light to the above behaviours. It would also be best to document the above.

Valory: There are several way to game karma, and we acknowledge that we need to include this into our documentation.

Cantina Managed: Acknowledged by Valory team.

3.4.3 Lack of two step ownership transfer pattern

Severity: Low Risk

Context: `Karma.sol#L80-L92`, `MechMarketplace.sol#L377-L390`

Description: The ownership transfer mechanism in `Karma.sol` and `MechMarketplace.sol` does not implement a two-step process, allowing ownership to be transferred in a single transaction without confirmation from the new owner.

A two-step approach ensures that ownership is only finalized once the intended recipient explicitly accepts it, reducing the risk of accidental transfers or loss of administrative control. Without this safeguard, ownership could be transferred to an address that is unable or unwilling to manage the contract, leading to operational challenges. Contracts affected:

- `changeOwner` function from `Karma.sol`.
- `changeOwner` function from `MechMarketplace.sol`.

Recommendation: Introducing a two-step ownership transfer process can enhance security by requiring the new owner to confirm acceptance before finalizing the transfer. This prevents unintended changes and ensures the designated recipient is aware and capable of assuming ownership responsibilities.

Valory: It is not necessary and will be problematic with our governance processing.

Cantina Managed: Acknowledged by Valory team.

3.4.4 Requests can be delivered non-sequentially

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: When a requester create a request, the request ids are calculated using the sequential nonce counter `mapNonces`. Afterwards the information regarding these nonces are lost/not used when mechs deliver the requests to the marketplace. So they could deliver these requests out of order. This is in contrast to the `deliverMarketplaceWithSignatures` endpoint where the sequential delivery of requests are enforced. For example if the requests are for a swapping services. And the user wants the service to perform swap S_1 before the swap S_2 then their order of execution would be important.

Recommendation: If above is based on the mechanism design it should be documented and shared with the users. If not:

1. `mapRequestIdInfos` should also store the nonce it was used to derived the request id.
2. Instead of just one mapping of `mapNonces` introduce two mappings to track requested/committed nonces per requester and also delivered nonces per requester.

The above would introduce an issue where one non-fulfilled request can block all proceeding requests. Specially if one also starts taking into consideration the priority mech in the calculation of the request id (that would block for a limited time only though).

Also in general the mechs can perform their services in a certain order but deliver the data to the market in a different order. So the true sequential guarantee even for the `deliverMarketplaceWithSignatures` cannot be enforced fully. There is a trust assumption there on the mechs. If this type of bad behaviours are noticed, the requester can stop using those mechs reducing their requester/mech karma acceleration. A monitoring system would also be recommended to pick up these bad behaviours.

Valory: Acknowledged.

Cantina Managed: Acknowledged by Valory team.

3.4.5 A mech calling `deliverMarketplaceWithSignatures` can use any payment type

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: When `deliverMarketplaceWithSignatures` is called, the payment type is not considered when checking the requester signatures against the derived request ids. This is due to the fact that request ids don't take into account the payment types. This would allow different payment type mechs fulfil/deliver the request which might not be desirable by the requester.

Recommendation: To avoid the above issue include the payment type also in the request id calculation. This would ensure that the requester would know about the payment type a priori to signing.

Valory: Fixed on PR 92 by adding `paymentType` to the id calculation.

Cantina Managed: Fix verified.

3.4.6 Free deliveries are only allowed when delivering with signature

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: When a mech does a simple delivery to the marketplace by calling `deliverMarketplace` in the relevant balance tracker there is the following check in `finalizeDeliveryRates` which prevents free deliveries:

```
// Check for zero value
if (totalMechDeliveryRate == 0) {
    revert ZeroValue();
}
```

The corresponding check is missing when calling `deliverMarketplaceWithSignatures` `adjustMechRequesterBalances`.

Recommendation: Either allow free deliveries in both routes or have the check also included in `deliverMarketplaceWithSignatures`.

Valory: Fixed on PR 93 by checking that `totalMechDeliveryRate` on `deliverMarketplaceWithSignatures` is not zero.

Cantina Managed: Fix verified.

3.4.7 Missing validation for balance tracker in `_requestBatch`

Severity: Low Risk

Context: `MechMarketplace.sol#L286`

Description: The function `_requestBatch` in `MechMarketplace.sol` retrieves the balance tracker contract from `mapPaymentTypeBalanceTrackers[paymentType]`. However, there is no validation ensuring that a valid contract address is assigned before usage. If this mapping has not been properly set, the contract will attempt to call `checkAndRecordDeliveryRates` on `address(0)`, leading to a revert.

While it is expected that the governance correctly configures payment types and their corresponding balance trackers, enforcing an explicit validation would improve contract robustness and avoid unintended reverts in edge cases.

Recommendation: To ensure contract stability, an explicit check can be added before calling functions on the balance tracker. If the mapping does not contain a valid contract address, the function could revert early with a clear error message, preventing unnecessary execution costs and improving error management.

Valory: Fixed on [PR 93](#).

Cantina Managed: Fix verified.

3.4.8 Delivering with signature does not check that `requestDatas[i]` should have a positive length

Severity: Low Risk

Context: [MechMarketplace.sol#L730](#), [MechMarketplace.sol#L252-L255](#)

Description: In the request/delivery flow the marketplace has the following check before deriving the request ids:

```
// Check for non-zero data
if (requestDatas[i].length == 0) {
    revert ZeroValue();
}
```

This check is missing in the delivery with signature flow.

Recommendation: To unify the flow check make sure either both flow skip this check or make sure `deliverMarketplaceWithSignatures` also has the above non-zero data check *before deriving the request id*. Perhaps this check can be moved to `getRequestId` to keep the logic atomic and avoid the above mistakes.

Valory: Fixed on [PR 93](#).

Cantina Managed: Fix verified.

3.5 Gas Optimization

3.5.1 memory can be replace by calldata to save gas

Severity: Gas Optimization

Context: [BalanceTrackerBase.sol#L192](#), [BalanceTrackerBase.sol#L231-L234](#), [BalanceTrackerBase.sol#L298-L299](#), [Karma.sol#L98](#), [MechFactoryBase.sol#L41](#), [MechFixedPriceBase.sol#L21](#), [MechFixedPriceBase.sol#L29](#), [MechMarketplace.sol#L211-L214](#), [MechMarketplace.sol#L312](#), [MechMarketplace.sol#L437](#), [MechMarketplace.sol#L463](#), [MechMarketplace.sol#L488](#), [MechMarketplace.sol#L524-L528](#), [MechMarketplace.sol#L577-L579](#), [MechMarketplace.sol#L697-L701](#), [OlasMech.sol#L216-L217](#), [OlasMech.sol#L268-L272](#)

Description: Several function parameters in the contracts use `memory` for input data that is not modified. Since `memory` incurs additional allocation costs, it can be replaced with `calldata` where applicable to reduce gas usage. Using `calldata` is particularly beneficial for external functions, as it avoids unnecessary copies and allows direct access to the transaction data.

The following occurrences were found:

- [Karma.sol](#) contract `setMechMarketplaceStatuses` function `mechMarketplaces` and `statuses` parameters.
- [MechFactoryBase.sol](#) contract `_createMech` function `payload` parameter.
- [MechFixedPriceBase.sol](#) contract `_preDeliver` function `data` parameter.
- [MechMarketplace.sol](#) contract `_requestBatch`, `_verifySignedHash`, `create`, `setMechFactoryStatuses`, `setPaymentTypeBalanceTrackers`, `request`, `requestBatch`, `deliverMarketplace`, `deliverMarketplaceWithSignatures` and `getRequestId` functions, nearly all parameters.
- [OlasMech.sol](#) contract `requestFromMarketplace` function.

Recommendation: Updating function parameters from `memory` to `calldata` where feasible can improve gas efficiency. Since `calldata` is immutable and cheaper in execution, adopting it where modification is not required ensures better gas optimization without affecting functionality.

Valory: Addressed on [PR 96](#).

Cantina Managed: Fix verified.

3.5.2 Owner role only used for initializing subscription parameters

Severity: Gas Optimization

Context: *(No context files were provided by the reviewer)*

Description: In `BalanceTrackerNvmSubscriptionNative.sol`, the owner is only utilized to set `subscriptionNFT` and `subscriptionTokenId`, but does not serve other administrative functions. If these values are determined at deployment and do not need to change, marking them as `immutable` could optimize gas costs and simplify contract logic.

Immutable variables are stored directly in bytecode rather than storage, reducing read and write operations. Additionally, this approach removes the need to manage and update ownership, reducing overall complexity and potential state modifications.

Recommendation: If `subscriptionNFT` and `subscriptionTokenId` are meant to remain constant after deployment, defining them as `immutable` in the constructor would enhance efficiency by reducing storage operations. This would also eliminate the need for setting and unsetting the owner, streamlining contract logic while saving gas.

Valory: Due to circular dependencies on deployment it is not possible to fix it. In order to register a subscription and make the contract its admin, the balance tracker contract needs to be deployed. Then, the NFT address and the subscription IDs are defined.

Cantina Managed: Acknowledged by Valory team.

3.5.3 Unused `paymentData` parameter

Severity: Gas Optimization

Context: [MechMarketplace.sol#L214](#)

Description: The `paymentData` parameter in the `_requestBatch` function of `MechMarketplace.sol` is included in the function signature but is not utilized in the function logic. This parameter is passed to subsequent function calls, including `checkAndRecordDeliveryRates` at line 289, which directs it to `BalanceTrackerBase.sol` at line 215. The function `_adjustInitialBalance`, where this parameter eventually arrives, does not make use of it. Additionally, in `BalanceTrackerNvmSubscriptionNative`, where `_adjustInitialBalance` is overridden, the parameter is still not used.

Similarly, in `deliverMarketplaceWithSignatures` at line 779, `paymentData` is passed to `adjustMechRequesterBalances`, which again routes it to `_adjustInitialBalance` without utilizing it. The presence of an unused parameter may indicate unnecessary complexity in the function signatures or a missing logic implementation that was originally intended.

Recommendation: If `paymentData` is meant to be used for processing payments or tracking balances, verifying its intended purpose and ensuring it is incorporated in relevant logic would be beneficial. If it is not required, removing it from function signatures could simplify the contract and reduce unnecessary data passing, optimizing gas efficiency.

Valory: This is left intentionally for any further possible cases where an initial payment data is needed, such that there is no need to change the MM ABI itself by upgrading it.

Cantina Managed: Acknowledged by Valory team.

3.6 Informational

3.6.1 Checking array lengths match is missing

Severity: Informational

Context: (No context files were provided by the reviewer)

Description/Recommendation: - [OlasMech.sol#L97-L98](#) - [BalanceTrackerBase.sol#L231-L234](#)

Valory: These functions are called internally or externally from upper-lever functions that have done these checks already.

Cantina Managed: The external ones rely on the assumption that the checks have been performed in the caller contracts. Currently it is fine, but in the future it might be something to be cautious about.

3.6.2 `getRequestStatus` can be simplified

Severity: Informational

Context: [MechMarketplace.sol#L847-L862](#)

Description/Recommendation: `getRequestStatus` can be simplified:

```
function getRequestStatus(bytes32 requestId) external view returns (RequestStatus status) {
    // Request exists if it has a record in the mapRequestIdInfos
    RequestInfo memory requestInfo = mapRequestIdInfos[requestId];

    if (requestInfo.priorityMech == address(0)) return RequestStatus.DoesNotExist;

    // Check if the request Id was already delivered: delivery mech address is not zero
    if (requestInfo.deliveryMech != address(0)) return RequestStatus.Delivered;

    if (block.timestamp > requestInfo.responseTimeout) {
        return RequestStatus.RequestedExpired;
    }

    return RequestStatus.RequestedPriority;
}
```

Valory: Fixed on PR 94.

Cantina Managed: Fix verified.

3.6.3 Use EIP-712 to derive request ids

Severity: Informational

Context: [MechMarketplace.sol#L801-L822](#)

Description/Recommendation:

1. Best to use EIP-712's typed structured data hashes to derive request ids.
2. add deadline as part of the typed structured data.

Cantina Managed: In [370e2b72099edf0b5407b342072ccfe0c8593354](#):

1. data has been hashed.
2. mech and paymentType are also included in request id derivation.

But the above recommendations are not applied.

Valory: Acknowledged.

Cantina Managed: Acknowledged by Valory team.

3.6.4 Canceling requests

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: It might make sense from the requester perspective to be able to cancel requests.

Recommendation: Canceling request feature can be added, but care needs to be taken into consideration since if a mech has already started working on a request it should be able to be compensated. Or else document for users that canceling requests are not possible directly.

Valory: Acknowledged as requests are not cancellable by design.

Cantina Managed: Acknowledged by Valory team.

3.6.5 Allow requesters to withdraw unused balances

Severity: Informational

Context: [BalanceTrackerBase.sol#L70](#).

Description: Currently, requesters can only deposit into the balance trackers. If there are any unused balances they would sit in this contract till they can be used for future requests. A workaround would be to:

1. The requester creates a mech and forms a fake request.
2. Then the requester delivers its own fake request and thus withdrawing its funds after paying the marketplace fee.

The only disincentive is that the requester would need to pay the marketplace fee.

Recommendation: Provide a functionality for users to withdraw any unused requester balances. The only drawback would be mechs who call the `deliverMarketplaceWithSignatures` with signatures as just before this call, the requester might pull out its funds from the balance tracker.

Valory: Acknowledged as it is a design decision. The protocol does not allow returns.

Cantina Managed: Acknowledged by Valory team matching its design constraints.

3.6.6 Unused code, Typos, Comments...

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description/Recommendation:

- [MechNvmSubscriptionNative.sol#L6-L18](#): The interface `IERC1155` is defined but not used.
- [Karma.sol#L48](#):

```
- external{  
+ external {
```

- [OlasMech.sol#L105-L117](#): Here the code implicitly assumes that the previous element of the newly added link node is 0:

```
mapRequestIds[requestId][0] == 0 // the `0` sentinel node
```

Currently this assumption is correct for freshly added request ids and the ones that have been previously cleared out. But perhaps adding a comment here to mention this assumption might be useful.

Valory: Fixed in commit [71b10e08](#).

Cantina Managed: Fix verified.

3.6.7 Implementation contracts can be initialized

Severity: Informational

Context: [MechMarketplace.sol#L364](#)

Description: The implementation contracts `Karma.sol` and `MechMarketplace.sol` allow direct initialization, which can result in their state being set independently of the proxy mechanism.

In upgradeable proxy patterns, the implementation contract is meant to remain uninitialized since the proxy is responsible for handling initialization. If an implementation contract exposes an initialization function without restriction, it can be initialized separately, potentially leading to a state that differs from the expected behavior of the deployed proxy.

Proof of Concept:

```
function test_implementationsCanBeInitialized() public {
    assertEq(MechMarketplaceProxy(payable(address(marketPlace))).getImplementation(), address(marketPlaceImpl));
    marketPlaceImpl.initialize(fee, minResponseTimeout, maxResponseTimeout);

    assertEq(karma.getImplementation(), address(karmaImpl));
    karmaImpl.initialize();
}
```

This test shows that the `initialize` function can be directly invoked on the implementation contracts, confirming that they do not enforce restrictions on direct initialization.

Recommendation: To ensure that implementation contracts cannot be initialized separately, a common approach is to introduce a protection mechanism that prevents their initialization outside the proxy. This can be achieved by adding a check that ensures the contract is being called through the proxy or by implementing an initialization guard that prevents repeated execution.

Valory: Calling `initialize()` outside the proxy context does not change the actual storage. Calling it only makes sense from a proxy contract.

Cantina Managed: Acknowledged by Valory team.

3.6.8 Burning or transferring subscriptionNFT credits should not be allowed

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Burning or transferring `subscriptionNFT` credits should not be allowed for the `BalanceTrackerNvmSubscriptionNative` to work properly.

- If transfer of credits allowed in `subscriptionNFT`, multiple requesters with different addresses can first buy credit, then create a request which will check to see whether they have enough credits, then transfer credits to a different address and create requests with the new address (without actually buying those credits). This operation can be done as many times as wanted and thus either cause mechs user other people's funds upon delivery or their delivery to the marketplace would fail since the `BalanceTrackerNvmSubscriptionNative` would not hold enough funds as native tokens. This bug currently present for the `MockNvmSubscriptionNative` implementation.
- The issue of burning credits is connected to the `redeemRequesterCredits` function since it would set the `mapRequesterBalances[requester]` to 0 for a requester and so if a mech delivers a data with a discounted delivery rate, the delivery function could revert due to `balance adjustment overflow`. Moreover the following statement would not always be true if users can directly call `subscriptionNFT` to burn their credits:

```
// This must never happen
if (subscriptionBalance < balance) {
    balance = subscriptionBalance;
}
```

On Gnosis chain the contract `0x1b5DeaD7309b56ca7663b3301A503e077Be18cba` which acts as `subscriptionNFT` can be called by credit holders to burn their own credits.

`BalanceTrackerNvmSubscriptionNative`'s implementation deviates from the other balance trackers for native and ERC20 tokens since in this case `mapRequesterBalances[requester]` holds the required credits by the requester for all the requests it has made vs for the other implementations where this parameter represents the unused amount of tokens (credits)(any used amount for native and token balance trackers would have been transferred to the tracker contract itself).

Recommendation:

1. Make sure transfers of `subscriptionNFT` are blocked or restricted to some privileged accounts.
2. The above rule could also apply to burn, unless some sort of extra functionality is needed there.

Or one could re-architecture the `BalanceTrackerNvmSubscriptionNative` so that `mapRequesterBalances[requester]` would represent unused credits.

- Upon requests the `subscriptionNFT` credits along with their underlying value (native tokens) should transfer to the `BalanceTrackerNvmSubscriptionNative`.

- And upon delivery of the request the corresponding credits should be burnt for `BalanceTrackerNvmSubscriptionNative` and underlying native tokens sent to the `mechs`. If there are any discounted deliveries, the discounted credit amount should be minted for the corresponding requesters (accounted using `mapRequesterBalances[requester]` and not `subscriptionNFT` credits).

Valory: Fixed in commit [e54d95d4](#).

Cantina Managed: In [e54d95d4](#), a different approach is taken where upon creating requests the amount of credits is either taken from:

- Any reimbursed discounted credit to the requester and then if there is any other left over credit that needs to be used.
- It is taken/burnt from the requester's balance on the net subscription contract.

In the above approach the transfer or burn of `subscriptionNFT` before or after creating a request does not affect the system since the required credits would be taken from the requester and they would not be able to for example double-spend.

3.6.9 To avoid checking array lengths for related arrays use structs

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Currently there are a few functions of the from below in the codebase:

```
function f(T1 memory a1[], T2 memory a2[], ...) ... {
    // check a1, a2, ... have the same length
}
```

Recommendation: To avoid the above checks and also save some gas on call data due to a more compact abi-encoding, instead use the following pattern:

```
struct A {
    T1 a1;
    T2 a2;
    // ...
}

function f(A memory a[]) ... {
    // no array length check is needed
}
```

moreover whenever possible use `calldata` instead of `memory` for the input parameter's location.

Valory: Fixed in commit [05b0db53](#).

Cantina Managed: Recommendation is partially applied in commit [05b0db53](#).

3.6.10 `mapRequestIdFinalizedRates` values cannot be trusted

Severity: Informational

Context: [MechNvmSubscriptionNative.sol#L34-L35](#)

Description: `mapRequestIdFinalizedRates` values cannot be trusted since if a request id has been delivered it can be delivered again by the `mech` but with a different delivery data (and so different delivery rates per request ids) but the delivered requests are *ignored/skipped* when getting processed in the market place. The effect would be that the value of `mapRequestIdFinalizedRates[k]` would be overridden and so it would not reflect the value that was used to process the original delivery.

Recommendation: `mapRequestIdFinalizedRates` is only used to glue the logic from `_preDeliver` to `getFinalizedDeliveryRates`. It might make sense to get ride of this parameter and instead combine the logic of these two hooks.

Valory: Fixed in commit [ef7d7e8a](#).

Cantina Managed: Fix verified.

3.6.11 Don't revert when a result is delivered by a different mech than the priority one during the response timeout period

Severity: Informational

Context: [MechMarketplace.sol#L645](#).

Description: In the context above we have the following revert statement:

```
    } else {  
        // Priority mech responseTimeout is still >= block.timestamp  
        revert PriorityMechResponseTimeout(requestInfo.responseTimeout, block.timestamp);  
    }
```

this will cause the whole delivery to the marketplace by the mech to fail if it provides a request id its responseTimeout has not expired.

Recommendation: If the revert is not necessary, one can change the above block to:

```
    } else {  
        continue;  
    }
```

Valory: Fixed on [PR 93](#).

Cantina Managed: Fix verified.

3.6.12 Mech operators can have the mech call any contract or perform any custom logic

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Mech operators can have the mech call any contract or perform any custom logic. This is due to the fact that `OlasMech` inherits from `Mech` which has the `exec` endpoint:

```
function _exec(  
    address to,  
    uint256 value,  
    bytes memory data,  
    Enum.Operation operation,  
    uint256 txGas  
) internal returns (bool success, bytes memory returnData) {  
    if (operation == Enum.Operation.DelegateCall) {  
        (success, returnData) = to.delegatecall{gas: txGas}(data);  
    } else {  
        (success, returnData) = to.call{gas: txGas, value: value}(data);  
    }  
}  
  
function exec(  
    address to,  
    uint256 value,  
    bytes memory data,  
    Enum.Operation operation,  
    uint256 txGas  
) public onlyOperator returns (bytes memory returnData) {  
    bool success;  
    (success, returnData) = _exec(  
        to,  
        value,  
        data,  
        operation,  
        txGas == 0 ? gasleft() : txGas  
    );  
  
    if (!success) {  
        // solhint-disable-next-line no-inline-assembly  
        assembly {  
            revert(add(returnData, 0x20), mload(returnData))  
        }  
    }  
}
```

thus the 'operator can change the values of any storage variables such as:

```
// Maximum required delivery rate
uint256 public maxDeliveryRate;
// Number of undelivered requests by this mech
uint256 public numUndeliveredRequests;
// Number of total requests by this mech
uint256 public numTotalRequests;
// Number of total deliveries by this mech
uint256 public numTotalDeliveries;
// Reentrancy lock
uint256 internal _locked = 1;

// Cyclical map of request Ids
mapping(bytes32 => bytes32[2]) public mapRequestIds;
```

which might not be useful, it might just trick some off or on-chain agents who would query for these values. Also it can perform custom function calls to the marketplace and avoid any implemented logic that is defined in the current contract. For example, the operator can make the mech call the marketplace delivery endpoint without the mech emitting the post call events:

```
// Traverse all requests to select delivered ones
for (uint256 i = 0; i < numRequests; ++i) {
    if (deliveredRequests[i]) {
        numDeliveries++;
        emit Deliver(address(this), msg.sender, requestIds[i], deliveryDatas[i]);
    } else {
        emit RevokeRequest(requestIds[i]);
    }
}
```

or perform external market calls without the reentrancy guard (can be reentered at least once).

Recommendation: Having the above open-ended endpoint should be analysed further. Currently it can also be used to transfer any tokens (native or not) or NFTs from the mech contract to another or make the mech contract call the balance tracker endpoint to process its payment.

Valory: This probably lacks any economical value. The mech statistics is not going to go anywhere, and is mostly held on MechMarketplace, so any ranking is going to be fetched from there. As for mapRequestIds, those are also present on the level of MM, and any manipulation there would most likely lead to reverts hurting the mech delivery.

Cantina Managed: Acknowledged by Valory team.

3.6.13 `setUp` function visibility can be restricted

Severity: Informational

Context: [OlasMech.sol#L280](#)

Description: The `setUp` function in `OlasMech.sol` is currently marked as `public`, even though it is only intended to be called during contract initialization. Since the function includes a safeguard that prevents multiple executions by reverting with `AlreadyInitialized()`, it does not introduce a risk of unintended state changes. However, keeping it `public` still allows external calls, which are unnecessary given that it is already executed from the constructor.

A function that is only meant to be called during initialization does not typically need to be externally accessible. If future implementations require the function to be overridden, making it `internal` would still allow modifications in derived contracts without exposing it externally. Otherwise, marking it `private` would ensure it is strictly used during deployment.

Recommendation: Restricting the visibility of `setUp` to `internal` or `private` would align with its intended use, preventing unnecessary external calls while still supporting flexibility in future contract extensions.

Valory: Fixed in [PR 101](#).

Cantina Managed: Fixed by Valory team. Now the code of the old `setUp` function is included in the constructor and the new `setUp` function does revert whenever is called.

3.6.14 Interface recommendations

Severity: Informational

Context: (No context files were provided by the reviewer)

Description/Recommendation:

- `IMech.sol#L15-L21`: `maxDeliveryRate` and `paymentType` can be defined as `view` this will guarantee that when those interface endpoints are used `staticcall` is made instead of a regular `call`.
- `IStaking.sol`: `IStaking.sol` does not seem to be used and perhaps can be removed.

Valory: Fixed on PR 94.

Cantina Managed: Fix verified.

3.6.15 When minting not subscription credit make sure the user always provides non-zero `msg.value`

Severity: Informational

Context: `MockNvmSubscriptionNative.sol#L30-L34`

Description: In the `MockNvmSubscriptionNative` implementation we have:

```
function mint(uint256 tokenId, uint256 numCredits) external payable {
    uint256 requiredAmount = numCredits * creditTokenRatio / 1e18;

    if (msg.value < requiredAmount) {
        revert InsufficientBalance(msg.value, requiredAmount);
    }

    _mint(msg.sender, tokenId, numCredits, "");
    // ...
}
```

Note that due to the division rounding errors the `requiredAmount` might be 0 and this the user can mint credits without paying any native tokens.

Recommendation: To avoid the above issue make sure:

1. `requiredAmount` is non-zero.
2. Best to round up `requiredAmount` in the formula `mulDivUp(numCredits, creditTokenRatio, 1e18)`.

Valory: Acknowledged.

Cantina Managed: Acknowledged by Valory team.

3.6.16 Best practices on signature verification process

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The `_verifySignedHash` function in `MechMarketplace.sol` does not fully implement best practices for signature verification. Specifically, the function does not apply signature malleability checks as recommended in EIP-2, which could allow multiple valid signatures for the same message by manipulating the `s` value.

Currently, OpenZeppelin's `ECDSA` library enforces stricter checks on the `s` value to ensure signatures are unique and follow best practices. The function should implement a similar validation to prevent potential issues.

Relevant best practices from OpenZeppelin's implementation include:

- Ensuring `s` is within the lower range to prevent malleability (`ECDSA.sol#L134-L145`).
- Optimizing `v` extraction using bitwise shifting (`ECDSA.sol#L108-L110`).

Despite the fact that each message hash is uniquely generated using a contract-side nonce, reinforcing signature verification according to best practices would enhance robustness and alignment with security standards.

Recommendation: To align with best practices, consider implementing additional checks to enforce signature uniqueness and prevent malleability. This includes validating the `s` value to ensure it falls within the lower range and optimizing `v` extraction using bitwise shifting. Additionally, marking the function as "memory-safe".

Valory: Noted. Not all recommendations are necessary in our case as the message cannot be repeated. Applied recommendations on [PR 97](#).

Cantina Managed: Fixes for `s` value applied as suggested.

3.6.17 Usage of modifiers

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The protocol manually implements reentrancy protection and owner checks within function bodies instead of defining and using Solidity modifiers. While function-level checks are effective, they introduce redundancy and increase the risk of inconsistencies if applied differently across functions.

Using explicit conditions within multiple functions results in repetitive code, making it harder to audit and maintain. A dedicated modifier for access control and another for reentrancy protection would ensure consistent enforcement while improving contract readability and maintainability.

Recommendation: Defining custom Solidity modifiers for owner verification and reentrancy protection can improve code clarity and reduce duplication. By centralizing these checks into reusable modifiers, the protocol ensures a uniform and maintainable approach while maintaining its preference for custom implementations.

Valory: Acknowledged.

Cantina Managed: Acknowledged by Valory team.

3.6.18 Consistent `getImplementation` function placement across proxies

Severity: Informational

Context: [Karma.sol#L153-L158](#), [MechMarketplaceProxy.sol#L74-L79](#)

Description: The `getImplementation` function is placed on different architectural contracts depending on the implementation. So, for the marketplace this function is placed on the proxy, `MechMarketplaceProxy`. However for the karma contract is placed over the implementation `Karma.sol` contract.

Recommendation: Consider placing the function either on both proxies or both implementations for consistency and clean code.

Valory: Fixed on [PR 100](#).

Cantina Managed: Fixed by Valory team.

3.6.19 Store `_serviceRegistry` and `_serviceId` as immutable parameters in the `OlasMech` contract

Severity: Informational

Context: [OlasMech.sol#L278-L299](#).

Description/Recommendation: Avoid storing `_serviceRegistry` and `_serviceId` in an external write-only-once contract and instead store them as immutable parameters in the `OlasMech` contract. `token()` and `tokenId()` can pull these values directly without performing a call to an external contract.

Valory: Fixed in commit [3b19fcf0](#).

Cantina Managed: Fixed in commit [3b19fcf0](#) using the recommendation.

3.6.20 Update responseTimeout and paymentType when delivering requests with signature

Severity: Informational

Context: (No context files were provided by the reviewer)

Context: MechMarketplace.sol#L743-L747

Description/Recommendation: Update responseTimeout and paymentType when delivering requests with signature. Currently only:

```
// Record all the request info
requestInfo.priorityMech = msg.sender;
requestInfo.deliveryMech = msg.sender;
requestInfo.requester = requester;
requestInfo.deliveryRate = deliveryRates[i];
```

The above are stored for these requests and when an entity queries for this request on/off-chain it would get 0 for both responseTimeout and paymentType.

Valory: Updated paymentType on PR 93, however keeping responseTimeout as zero would clearly separate that requests were delivered with signatures.

Cantina Managed: Fixed by Valory team by updating paymentType.

3.6.21 _verifySignedHash does not catch incorrect v values early

Severity: Informational

Context: (No context files were provided by the reviewer)

Context: MechMarketplace.sol#L335-L341.

Description: In this context we have:

```
// For the correct ecrecover() function execution, the v value must be set to {0,1} + 27
// Although v in a very rare case can be equal to {2,3} (with a probability of 3.73e-37%)
// If v is set to just 0 or 1 when signing by the EOA, it is most likely signed by the ledger and must be
↳ adjusted
if (v < 4) {
    // In case of a non-contract, adjust v to follow the standard ecrecover case
    v += 27;
}
```

_verifySignedHash allows for v values to be 2 and 3 when passed to ecrecover. Although in that case solc's ecrecover would return address(0) which will cause below to revert since we know that requester cannot be address(0):

```
// Final check is for the requester address itself
if (recRequester != requester) { // <--- here `recRequester` would be `address(0)`
    revert SignatureNotValidated(requester, requestHash, signature);
}
```

Proof of Concept: Add the following test contract and test file contracts/test/TestVerifySig.sol:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

contract TestVerifySig {
    function getRequester(
        bytes32 requestHash,
        bytes memory signature
    ) external pure returns (address) {
        // Decode the signature
        uint8 v = uint8(signature[64]);

        // For the correct ecrecover() function execution, the v value must be set to {0,1} + 27
        // Although v in a very rare case can be equal to {2,3} (with a probability of 3.73e-37%)
        // If v is set to just 0 or 1 when signing by the EOA, it is most likely signed by the ledger and must
        ↪ be adjusted
        if (v < 4) {
            // In case of a non-contract, adjust v to follow the standard ecrecover case
            v += 27;
        }

        bytes32 r;
        bytes32 s;
        // solhint-disable-next-line no-inline-assembly
        assembly {
            r := mload(add(signature, 32))
            s := mload(add(signature, 64))
        }

        // Case of ecrecover with the request hash for EOA signatures
        return ecrecover(requestHash, v, r, s);
    }
}

```

test/TestVerifySig.js:

```

/*global describe, context, beforeEach, it*/

const { expect } = require("chai");
const { config, ethers } = require("hardhat");

function randomSig(digest, v) {
    const accounts = config.networks.hardhat.accounts;
    const wallet = ethers.Wallet.fromMnemonic(accounts.mnemonic, accounts.path + `/${0}`);
    const signingKey = new ethers.utils.SigningKey(wallet.privateKey);

    const signature = signingKey.signDigest(digest);
    // Extract v, r, s
    const r = ethers.utils.arrayify(signature.r);
    const s = ethers.utils.arrayify(signature.s);
    const vMod = ethers.utils.arrayify(v);
    // Assemble 65 bytes of signature
    return ethers.utils.hexlify(ethers.utils.concat([r, s, vMod]));
}

describe("TestVerifySig", function () {
    let testVerifySig;
    const AddressZero = ethers.constants.AddressZero;

    beforeEach(async function () {
        testVerifySig = await ethers.getContractFactory("TestVerifySig");
        testVerifySig = await testVerifySig.deploy();
        await testVerifySig.deployed();
    });

    context("signature verification", async function () {
        it("v: 2", async function () {
            const digest = ethers.utils.keccak256(ethers.utils.toUtf8Bytes("test"));
            const signature = randomSig(digest, 2);
            const requester = await testVerifySig.getRequester(
                digest,
                signature
            );

            expect(requester).to.be.equal(AddressZero);
        });
    });
});

```

```

it("v: 3", async function () {
  const digest = ethers.utils.keccak256(ethers.utils.toUtf8Bytes("test"));
  const signature = randomSig(digest, 3);
  const requester = await testVerifySig.getRequester(
    digest,
    signature
  );

  expect(requester).to.be.equal(AddressZero);
});
it("v: 0", async function () {
  const digest = ethers.utils.keccak256(ethers.utils.toUtf8Bytes("test"));
  const signature = randomSig(digest, 0);
  const requester = await testVerifySig.getRequester(
    digest,
    signature
  );

  expect(requester).to.be.not.equal(AddressZero);
});
it("v: 1", async function () {
  const digest = ethers.utils.keccak256(ethers.utils.toUtf8Bytes("test"));
  const signature = randomSig(digest, 1);
  const requester = await testVerifySig.getRequester(
    digest,
    signature
  );

  expect(requester).to.be.not.equal(AddressZero);
});
});
});

```

And then run `yarn run hardhat test -- --grep "TestVerifySig"`.

Recommendation: If you want to catch the incorrect `v` values early, if the provided `v` values is 2 or 3 revert.

Note: `geth` would return `nil` or empty return data for invalid values of `v`:

```

package main

import (
  "fmt"
  "github.com/ethereum/go-ethereum/common"
  "github.com/ethereum/go-ethereum/core/vm"
)

func main() {
  ecr := vm.PrecompiledContractsPrague[common.BytesToAddress([]byte{0x1})]

  h := randBytes(32)
  rs := randBytes(64)

  input := make([]byte, 128)
  copy(input, h)
  copy(input[64:128], rs)
  input[63] = byte(3+27) // <--- v = 3

  addr, err := ecr.Run(input)
  if err != nil {
    fmt.Errorf(err.Error())
  }

  fmt.Println(addr)
}

```

but in `solc` those are returned as `address(0)`.

3.6.22 No blocking mechanism for malicious mechs in the marketplace exists

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Currently, if a mech starts to misbehave after its creation there is no mechanism to block its usage both in creating requests or delivery.

Recommendation: If the above blocking mechanism is required, in the Mech marketplace, perhaps the admin/owner should be able to block mechs by their address or block the whole group of mechs by their service id, so:

1. Requesters would not be able to create requests with those mechs.
2. Those mechs would not be able to deliver any created requests.